

LAPORAN TUGAS
PROGRAMMING LANGUAGE PRAGMATICS
ANALISIS KUALITAS LIMA BAHASA PEMROGRAMAN
BERDASARKAN ISO/IEC 25010:2023

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Programming Language Pragmatics



Disusun Oleh :
Nama : Ghaisan Khoirul Badruzaman
NIM : 251524048
Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

BAB I PENDAHULUAN

1.1 Tujuan dan Pilihan Bahasa

Tugas ini menjawab pertanyaan pada slide Suplemen 1 PLP: apakah lima bahasa pemrograman memenuhi karakteristik kualitas pada Gambar 1, lengkap dengan justifikasi dan kasusnya. Bahasa yang dipilih adalah C, Java, Python, JavaScript, dan SQL. Kelimanya disebut langsung oleh slide 6 dan slide 9, jadi posisinya di taksonomi bisa dirujuk ke materi tanpa perlu menebak. Kombinasi ini juga mencakup tiga level abstraksi (middle-level, high-level, very high-level) dan empat dari lima paradigma, sehingga perbedaan vonis antarbahasa bisa dilacak ke perbedaan desainnya.

1.2 Karakteristik Kualitas pada Gambar 1

Gambar 1 pada slide adalah model kualitas produk ISO/IEC 25010:2023. Dibanding ISO 9126 yang juga ditampilkan di materi, security dan interoperability yang dulu cuma sub-karakteristik functionality kini berdiri sendiri sebagai Security dan Compatibility, usability berganti nama menjadi Interaction Capability, portability menjadi Flexibility, dan Safety masuk sebagai karakteristik baru di revisi 2023. Tabel 1.1 merangkum kesembilan karakteristiknya.

Tabel 1.1: Karakteristik kualitas ISO/IEC 25010:2023

Karakteristik	Arti singkat	Sub-karakteristik
Functional Suitability	Fungsi yang disediakan lengkap, benar, dan sesuai kebutuhan	functional completeness, functional correctness, functional appropriateness
Performance Efficiency	Kinerja dibanding waktu dan sumber daya yang dipakai	time behaviour, resource utilization, capacity
Compatibility	Bisa berbagi lingkungan dan bertukar data dengan sistem lain	co-existence, interoperability
Interaction Capability	Mudah dipelajari dan dipakai penggunaanya, dalam tugas ini programmer	appropriateness recognizability, learnability, operability, user error protection, user engagement, inclusivity, user assistance, self-descriptiveness
Reliability	Berjalan benar dan tetap bisa pulih saat terjadi gangguan	faultlessness, availability, fault tolerance, recoverability
Security	Melindungi data dan sistem dari akses yang tidak sah	confidentiality, integrity, non-repudiation, accountability, authenticity, resistance
Maintainability	Mudah dipahami, diubah, dan diuji	modularity, reusability, analysability, modifiability, testability
Flexibility	Mudah dipindah atau disesuaikan ke lingkungan lain	adaptability, scalability, installability, replaceability
Safety	Tidak membahayakan manusia, harta, atau lingkungan	operational constraint, risk identification, fail safe, hazard warning, safe integration

1.3 Cara Menilai Bahasa dengan Standar Produk

Ada masalah mendasar sebelum mulai menilai. ISO/IEC 25010 dibuat untuk produk perangkat lunak, bukan untuk bahasa pemrograman. Bahasa tidak bisa crash atau lambat dengan sendirinya, yang crash dan lambat adalah program yang ditulis dengan bahasa itu. Supaya penilaiannya tidak asal, tiap vonis di tugas ini menyebut lapisan mana yang sebenarnya dinilai:

1. **Desain bahasa**, yaitu aturan sintaks dan semantik di spesifikasinya, misalnya aturan tipe data atau cara bahasa menangani memori.
2. **Implementasi**, yaitu compiler, interpreter, atau runtime yang menjalankan program, misalnya GCC, JVM, CPython, V8, atau mesin DBMS.
3. **Ekosistem**, yaitu library, package manager, dan tooling di sekitar bahasa, misalnya npm, pip, Maven, atau linter.

Soal memakai kata "memenuhi", jadi jawabannya dibuat harfiah dengan tiga vonis pada Tabel 1.2. Vonis Bersyarat bukan jawaban aman untuk semua baris. Vonis ini hanya dipakai kalau memang ada syarat konkret yang bisa disebut.

Tabel 1.2: Skala vonis

Vonis	Arti
Ya	Desain bahasa atau implementasi resminya menjamin karakteristik itu tanpa bergantung pada disiplin programmer.
Bersyarat	Karakteristik bisa tercapai, tetapi butuh library, alat tambahan, konfigurasi, atau disiplin programmer.
Tidak	Desain bahasa justru mempersulit karakteristik itu tercapai.

1.4 Kaitan dengan Kriteria Evaluasi Bahasa di Slide

Slide 23 sampai 27 sudah memberi kriteria evaluasi bahasa versi Sebesta. Kriteria itu ternyata bisa dipetakan ke ISO/IEC 25010, dan pemetaan ini yang dipakai sebagai jembatan saat menilai bahasa dengan standar produk (Tabel 1.3). Hasil pemetaannya juga menunjukkan kekurangan kriteria Sebesta: Security dan Safety tidak punya padanan langsung, padahal banyak kasus paling mahal di BAB II justru ada di dua karakteristik itu.

Tabel 1.3: Pemetaan kriteria Sebesta ke ISO/IEC 25010:2023

Kriteria Sebesta	Karakteristik ISO (sub)	Alasan
Readability	Maintainability (analysability), Interaction Capability (learnability)	Kode yang mudah dibaca lebih cepat dianalisis saat dirawat dan lebih cepat dipelajari programmer baru.
Writability	Interaction Capability (operability), Functional Suitability (functional appropriateness)	Ekspresivitas dan dukungan abstraksi menentukan seberapa pas bahasa dipakai untuk menulis solusi di domainnya.

Kriteria Sebesta	Karakteristik ISO (sub)	Alasan
Reliability	Reliability (faultlessness, fault tolerance)	Type checking dan exception handling mencegah atau menahan kesalahan saat program berjalan.
Cost	Performance Efficiency (time behaviour, resource utilization), Interaction Capability (learnability)	Biaya eksekusi setara efisiensi kinerja, sedangkan biaya pelatihan bergantung pada seberapa mudah bahasa dipelajari.
Portability	Flexibility (adaptability, installability)	Program yang mudah dipindah antarplatform berarti mudah diadaptasi dan dipasang di lingkungan lain.
Generality	Functional Suitability (functional completeness)	Bahasa yang umum bisa dipakai di banyak domain, sedangkan DSL sengaja lengkap di satu domain saja.
Well-definedness	Compatibility (interoperability)	Definisi resmi yang presisi membuat implementasi berbeda memberi hasil yang sama untuk kode yang sama.
(tidak ada)	Security, Safety	Sebesta hanya menyinggung aliasing dan type checking sebagai bagian reliability, belum sebagai kriteria sendiri.

1.5 Posisi Lima Bahasa di Taksonomi Slide

Tabel 1.4: Level abstraksi dan paradigma kelima bahasa

Bahasa	Level (slide 6)	Paradigma (slide 9 dan 12)	Contoh produk nyata
C	Middle-level	Imperative/procedural	Kernel Linux, SQLite
Java	High-level	OOP berbasis class, multi-paradigm sejak lambda Java 8	Minecraft: Java Edition, banyak aplikasi Android
Python	High-level	Multi-paradigm, OOP dominan	Instagram (Django), skrip pengolahan data
JavaScript	High-level	Event-driven (event loop), multi-paradigm	Halaman web interaktif, server Node.js
SQL	Very high-level/4GL, sekaligus DSL untuk data	Declarative	Query di PostgreSQL, MySQL, SQLite

BAB II ANALISIS LIMA BAHASA

2.1 C

C masuk kelompok middle-level (slide 6) bersama C++, karena pointer dan operasi bitnya dekat ke mesin tetapi kodenya tetap bisa dipindah antar arsitektur. Paradigmanya imperative/procedural (slide 9), dan Ritchie sendiri menggolongkan C ke keluarga procedural bersama Fortran dan Algol 60, sehingga C bukan contoh bahasa multi-paradigm seperti di slide 12.

Tabel 2.1: Penilaian C terhadap ISO/IEC 25010:2023

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Functional Suitability functional completeness, functional appropriateness (desain bahasa)	Bersyarat	Pointer, operasi bit, dan struct cukup untuk pemrograman sistem: kernel Unix ditulis ulang dalam C pada 1973, dan kernel Linux sampai sekarang ditulis dalam C. Tapi C tidak lengkap sendirian. Thompson (1978) mencatat kernel Unix berisi sekitar 10.000 baris C dan 1.000 baris assembly, 800 di antaranya untuk fungsi hardware yang tidak bisa ditulis dalam C.
Performance Efficiency time behaviour, resource utilization (implementasi)	Ya	GCC dan Clang mengompilasi C langsung ke kode mesin, dan tidak ada garbage collector (pembersih memori otomatis) yang ikut berjalan. Pada pengukuran 27 bahasa oleh Pereira dkk. (SLE 2017), C peringkat 1 untuk energi dan waktu serta peringkat 3 untuk memori, sedangkan Python butuh 75,88 kali energi dan 71,90 kali waktu C. Yang dibandingkan adalah versi program tercepat di tiap bahasa.
Compatibility interoperability, co-existence (implementasi)	Ya	Tim SQLite memilih C karena hampir semua sistem bisa memanggil library C, sehingga SQLite yang sama dipakai aplikasi Android berbahasa Java maupun aplikasi iPhone berbahasa Objective-C atau Swift. Versi minimumnya hanya butuh tujuh fungsi pustaka standar C, tanpa runtime tambahan seperti JVM.
Interaction Capability user error protection (desain bahasa)	Tidak	Percobaan backdoor kernel Linux 2003 menyamarkan <code>current->uid = 0</code> sebagai pengecekan, padahal satu tanda = itu mengubah pemanggilnya menjadi root (hak akses tertinggi). Kodenya sah karena C membolehkan penugasan dipakai sebagai kondisi. GCC 15.2 dan Clang 23.1 dengan <code>-Wall -Wextra</code> memang memperingatkan <code>if (x = 0)</code> biasa, tetapi meloloskan pola backdoor tersebut.
Reliability faultlessness, fault tolerance (desain bahasa)	Bersyarat	Pada 15 Januari 1990 jaringan telepon jarak jauh AT&T lumpuh sekitar sembilan jam karena satu <code>break</code> di program sentral 4ESS berbahasa C. Maksudnya keluar dari <code>if</code> , tetapi yang dihentikan justru <code>switch</code> , lalu gangguannya merambat ke 114 sentral. Program C bisa sangat andal lewat pengujian dan review, tetapi keandalannya tidak dijamin bahasanya.
Security confidentiality (desain bahasa)	Tidak	Pada Heartbleed (CVE-2014-0160) di OpenSSL, panjang data dari paket penyerang dipakai <code>memcpy</code> tanpa dicek, sehingga server membocorkan sampai 64 KB isi memorinya, termasuk private key. C tidak mengecek batas buffer, sampai NSA (2022) menyarankan organisasi mempertimbangkan beralih ke bahasa memory safe bila memungkinkan.

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Maintainability analysability, modifiability (desain bahasa)	Bersyarat	Makro C hanya diganti teksnya oleh preprocessor, sehingga <code>KUADRAT(1+2)</code> pada kode di bawah menjadi <code>1+2*1+2</code> dan bernilai 5, bukan 9. Manual GNU CPP menyarankan tiap parameter makro dibungkus kurung, jadi kode C tetap mudah dianalisis asal konvensi seperti itu dipatuhi.
Flexibility adaptability (desain bahasa)	Bersyarat	Standar C menyerahkan ukuran tipe ke implementasi: <code>sizeof(long)</code> bernilai 8 byte di macOS arm64 (dicoba dengan Apple clang 17) tetapi 4 byte di MSVC untuk Windows 64-bit. Angka besar yang disimpan di <code>long</code> bisa benar di Mac lalu salah di Windows, kecuali programmer memakai tipe berlebar pasti seperti <code>int64_t</code> .
Safety fail safe, operational constraint (ekosistem)	Bersyarat	C tidak menghentikan akses memori yang salah, jadi kondisi fail safe harus diusahakan dari luar bahasa. Contohnya aturan The Power of 10 (Holzmann, IEEE Computer 2006) yang diuji coba NASA/JPL untuk software misi kritis berbahasa C: tiap loop wajib punya batas atas tetap, dan kode harus bersih dari warning compiler serta static analyzer setiap hari.

Kode 2.1: contoh untuk baris Interaction Capability pada C

```
if ((options == (__WCLONE|__WALL)) && (current->uid = 0))
    retval = -EINVAL;
```

Kode 2.2: contoh untuk baris Maintainability pada C

```
#define KUADRAT(x) x*x
printf("%d\n", KUADRAT(1+2)); /* 5, bukan 9 */
```

Rangkuman C: C paling cocok untuk produk yang memprioritaskan kecepatan, hemat memori, dan mudah dipanggil dari bahasa lain, seperti kernel sistem operasi atau library semacam SQLite, asalkan keamanan memorinya dijaga dengan aturan koding yang ketat.

2.2 Java

Slide 6 menaruh Java di high-level dengan paradigma dominan OOP berbasis class (slide 9), tetapi sesuai slide 12 Java juga multi-paradigm sejak lambda (fungsi tanpa nama) di Java 8 (JEP 126) membuka gaya fungsional. Kodenya dikompilasi `javac` menjadi bytecode lalu dijalankan JVM, jadi sebagian kualitas Java sebenarnya datang dari JVM, bukan dari bahasanya.

Tabel 2.2: Penilaian Java terhadap ISO/IEC 25010:2023

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Functional Suitability functional correctness (desain bahasa)	Bersyarat	Menurut JLS §4.2.2, overflow <code>int</code> terjadi tanpa tanda apa pun. Akibatnya <code>low + high</code> pada rumus titik tengah <code>Arrays.binarySearch</code> bisa bernilai negatif untuk array berisi 2^{30} elemen ke atas, bug JDK-5045582 yang tersembunyi sekitar sembilan tahun sebelum diperbaiki di Java 6. Hasil yang benar baru terjamin kalau programmer ingat memakai <code>long</code> atau <code>Math.addExact</code> .

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Performance Efficiency time behaviour (implementasi)	Bersyarat	Di One Billion Row Challenge (Januari 2024), program pembanding sederhana di OpenJDK 21 butuh 4 menit 49 detik untuk mengolah 1 miliar baris data suhu, sedangkan pemenangnya hanya 1,535 detik dengan GraalVM native image (dikompilasi langsung ke kode mesin) dan <code>sun.misc.Unsafe</code> (akses memori tanpa pengecekan JVM). Java bisa sangat cepat, tetapi hasilnya bergantung pada runtime, konfigurasi, dan cara kode ditulis.
Compatibility interoperability (ekosistem)	Bersyarat	Java SE sudah punya <code>HttpClient</code> standar sejak JDK 11 (JEP 321), tetapi API JSON bawaan (JEP 540) baru masuk JDK 28 yang masih dikembangkan, itu pun sebagai modul incubator yang harus diaktifkan manual. Jadi aplikasi Java yang membaca respons REST API praktis masih perlu library pihak ketiga seperti Jackson atau Gson.
Interaction Capability learnability (desain bahasa)	Bersyarat	JEP 512 yang final di JDK 25 membolehkan program pertama ditulis tanpa <code>class</code> , <code>public static</code> , <code>String[] args</code> , maupun <code>import</code> , seperti kode di bawah, dengan tujuan resmi memberi "smooth on-ramp" bagi pemula. Namun kemudahan ini butuh JDK 25 ke atas dan hanya terasa di program kecil, karena begitu program membesar konsep <code>class</code> dan <code>import</code> tetap harus dipelajari.
Reliability faultlessness (desain bahasa)	Bersyarat	<code>null</code> boleh diisikan ke tipe referensi apa pun (JLS §4.1), sehingga <code>nama.length()</code> pada <code>String nama = null</code> lolos kompilasi lalu crash dengan <code>NullPointerException</code> (NPE) saat dijalankan. Meta sampai membuat static analyzer <code>Nullsafe</code> , dan selama 18 bulan penerapannya crash NPE aplikasi Android Instagram turun 27%, walaupun Meta mengakui penurunan itu juga dipengaruhi upaya lain.
Security integrity, resistance (ekosistem)	Bersyarat	Akses array Java selalu dicek batasnya saat runtime (JLS §10.4), jadi buffer overflow tidak terjadi di kode Java biasa, dan NSA (2022) memasukkan Java ke daftar bahasa memory safe. Namun Log4Shell (CVE-2021-44228, CVSS 10) di library Apache Log4j 2 membuat penyerang bisa menjalankan kode dari jarak jauh, di Minecraft: Java Edition cukup lewat pesan chat. Keamanan aplikasi Java jadi ikut ditentukan library-nya.
Maintainability analysability, modifiability (desain bahasa)	Ya	Karena bertipe statis, mengubah parameter method dari <code>int</code> menjadi <code>String</code> langsung memunculkan error kompilasi di semua pemanggil yang belum disesuaikan, jadi dampak perubahan terlihat sebelum program jalan. Sejak Java 16 (JEP 395), <code>record Mahasiswa(String nim, String nama) {}</code> membuat compiler menyusun constructor, accessor, <code>equals</code> , <code>hashCode</code> , dan <code>toString</code> , yang ikut menyesuaikan saat field ditambah.
Flexibility adaptability (implementasi)	Ya	Bytecode dalam class file tidak terikat hardware maupun OS (JVMS §2.1), jadi <code>.jar</code> berisi bytecode murni bisa dijalankan di OS mana pun yang punya JVM tanpa kompilasi ulang. Minecraft: Java Edition memakai satu <code>client.jar</code> untuk Windows, macOS, dan Linux, tetapi library native LWJGL-nya tetap dipisah per OS, jadi jaminan ini tidak berlaku untuk bagian kode native.
Safety operational constraint, safe integration (desain bahasa)	Tidak	Memori objek Java hanya dibersihkan garbage collector dan class-nya dimuat JVM secara dinamis, padahal JSR 302 Safety Critical Java mencatat kebanyakan standar safety-critical menuntut alokasi resource yang pasti sejak awal, sehingga GC dan class yang dimuat dinamis mungkin tidak bisa dipakai. JSR 302 ditujukan untuk sistem yang bisa lolos sertifikasi seperti DO-178B, tetapi sejak 2006 belum mencapai Final Release.

Kode 2.3: contoh untuk baris Suitability pada Java

```
int low = 1_500_000_000, high = 1_600_000_000;  
System.out.println((low + high) / 2); // -597483648  
System.out.println((low + high) >>> 1); // 1550000000, cara perbaikan di JDK
```

Kode 2.4: contoh untuk baris Interaction Capability pada Java

```
void main() {  
    IO.println("Halo, PLP!");  
}
```

Rangkuman Java: Java paling cocok untuk produk yang memprioritaskan maintainability dan flexibility, seperti aplikasi bisnis atau server yang dirawat tim bertahun-tahun dan harus jalan di berbagai OS, tetapi tidak untuk sistem safety-critical seperti software pesawat.

2.3 Python

Slide 6 menaruh Python di high-level, bukan very high-level seperti sebutan tutorial resminya, karena slide memakai istilah itu untuk bahasa deklaratif seperti SQL. Slide 9 dan 12 menjadikannya contoh multi-paradigm dengan OOP dominan, sebab semua nilai berupa objek, walaupun program tetap boleh ditulis prosedural tanpa class.

Tabel 2.3: Penilaian Python terhadap ISO/IEC 25010:2023

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Functional Suitability functional completeness, functional correctness (desain bahasa)	Bersyarat	<code>int</code> tidak punya batas selama memori cukup, jadi <code>2**100</code> langsung tepat tanpa overflow, dan <code>json</code> , <code>csv</code> , serta <code>sqlite3</code> sudah tersedia di pustaka standar. Tapi <code>float</code> hanya mendekati angka desimal, sehingga <code>0.1 + 0.1 + 0.1 == 0.3</code> bernilai <code>False</code> , dan hitungan uang baru tepat kalau programmer sengaja memakai modul <code>decimal</code> .
Performance Efficiency time behaviour, resource utilization (implementasi)	Bersyarat	GIL (Global Interpreter Lock) di CPython mencegah beberapa thread menjalankan kode Python bersamaan, sehingga CPU multi-core tidak terpakai efisien (PEP 703), dan angka 1 saja memakan 28 byte karena berupa objek. Versi tanpa GIL baru didukung resmi di 3.14 dan masih opsional, jadi program yang butuh cepat masih bergantung pada multiprocessing atau library berinti C seperti NumPy.
Compatibility co-existence, interoperability (ekosistem)	Bersyarat	Modul bawaan <code>ctypes</code> bisa memanggil library C (interoperability), tetapi <code>pip install global</code> bisa menimpa paket Python milik OS (co-existence). Karena itu, mengikuti PEP 668, Python sistem di Debian 12, Ubuntu 23.04 ke atas, dan Python 3.12 ke atas dari Homebrew menolak <code>pip install global</code> dengan error <code>externally-managed-environment</code> , sehingga tiap proyek perlu <code>venv</code> .
Interaction Capability learnability, self-descriptiveness (implementasi)	Ya	Pesan error CPython versi baru sering langsung menunjuk penyebabnya: kurung yang lupa ditutup dijawab <code>'(' was never closed</code> sejak 3.10, <code>print(sys.argv)</code> tanpa <code>import</code> dijawab <code>Did you forget to import 'sys'?</code> sejak 3.12, dan salah ketik <code>while</code> dijawab <code>Did you mean 'while'?</code> sejak 3.14. Semuanya bawaan interpreter resmi, tanpa library tambahan.

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Reliability faultlessness, fault tolerance (desain bahasa)	Bersyarat	Blok <code>try</code> , <code>except</code> , dan <code>finally</code> cukup untuk fault tolerance, tetapi faultlessness bergantung disiplin programmer karena aliasing (dua nama untuk satu objek) terjadi diam-diam. Pada kode di bawah, list default dibuat sekali saat <code>def</code> dijalankan dan hidup selama fungsinya ada (binding dan lifetime, materi praktikum 1), sehingga dua panggilan berbagi list yang sama tanpa error.
Security integrity, resistance (implementasi)	Bersyarat	CISA (2023) menggolongkan Python sebagai memory safe language, tetapi pustaka standarnya tetap bisa jadi celah: CVE-2007-4559 membuat <code>tarfile.extractall</code> bisa menulis file yang namanya memuat <code>../</code> ke luar folder tujuan, dan Trellix (2022) memperkirakan lebih dari 350.000 proyek open source berpotensi rentan. Filter pengamanannya baru menjadi default di Python 3.14 (PEP 706).
Maintainability analysability (ekosistem)	Bersyarat	Type hint tidak dicek saat program berjalan (PEP 484), jadi kode di bawah mencetak 12 tanpa error walaupun parameternya ditandai <code>int</code> . Dropbox (2019) menulis bahwa tanpa anotasi, sekadar tahu argumen yang valid untuk sebuah fungsi saja sudah sulit, sehingga mereka memakai pengecek tipe <code>mypy</code> sampai hampir 4 juta baris Python-nya tercek.
Flexibility adaptability (implementasi)	Bersyarat	Skrip <code>.py</code> tidak perlu dikompilasi ulang per OS, dan Windows, macOS, serta Linux ada di Tier 1 CPython (PEP 11). Tapi sampai 3.14, <code>open()</code> tanpa argumen <code>encoding</code> memakai encoding bawaan OS, bukan selalu UTF-8. Menurut PEP 597, kelalaian ini membuat 82 dari 4.000 paket PyPI terpopuler gagal dipasang di sistem yang encoding bawaannya bukan UTF-8. UTF-8 baru jadi default di 3.15 (PEP 686).
Safety operational constraint, fail safe (desain bahasa)	Bersyarat	Python gagal dengan aman: <code>nilai[5]</code> pada list tiga elemen langsung memunculkan <code>IndexError</code> , bukan membaca memori acak. Tapi batas operasi yang ditulis sebagai <code>assert suhu < 100</code> lenyap saat program dijalankan dengan <code>python -O</code> , jadi pengaman penting harus ditulis dengan <code>if</code> dan <code>raise</code> .

Kode 2.5: contoh untuk baris Reliability pada Python

```
def tambah_nilai(n, daftar=[]):
    daftar.append(n)
    return daftar
print(tambah_nilai(1), tambah_nilai(2)) # [1, 2] [1, 2], bukan [1] [2]
```

Kode 2.6: contoh untuk baris Maintainability pada Python

```
def tambah(a: int, b: int) -> int:
    return a + b
print(tambah("1", "2")) # 12
```

Rangkuman Python: Python paling cocok untuk produk yang memprioritaskan kemudahan dipelajari, pustaka bawaan yang lengkap, dan jalan di banyak OS tanpa kompilasi ulang, seperti skrip otomasi, pengolahan data, dan prototipe, bukan sistem yang menuntut performa multi-core atau batas keselamatan yang ketat.

2.4 JavaScript

Slide 6 menaruh JavaScript di high-level, dan slide 9 memakai event loop-nya sebagai contoh paradigma event-driven, walaupun event loop itu sebenarnya diatur spesifikasi HTML di browser dan libuv di Node.js, bukan oleh ECMAScript, standar resmi bahasanya. Sesuai slide 12, JavaScript lebih tepat disebut multi-paradigm: berbasis prototype, dan mendukung gaya imperatif, fungsional, serta berorientasi objek.

Tabel 2.4: Penilaian JavaScript terhadap ISO/IEC 25010:2023

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Functional Suitability functional correctness, functional completeness (desain bahasa)	Bersyarat	Tipe <code>number</code> adalah floating point 64-bit (IEEE 754), jadi bilangan bulat di atas $2^{53} - 1$ tidak lagi dijamin tepat dan bisa berubah tanpa error, seperti pada kode di bawah. Dokumentasi X (dulu Twitter) sampai meminta developer memakai <code>id_str</code> karena ID 64-bit kehilangan presisi di JavaScript. Angka yang tepat baru didapat kalau programmer memilih <code>BigInt</code> (ES2020).
Performance Efficiency time behaviour, resource utilization (implementasi)	Bersyarat	Event loop Node.js bisa melayani banyak klien dengan satu thread karena operasi I/O diserahkan ke kernel, tetapi kerja CPU yang berat membuat semua klien lain ikut menunggu. Panduan resmi Node.js mencontohkan <code>JSON.parse</code> string 50 MB yang butuh 1,3 detik, dan selama itu server tidak melayani request lain, kecuali programmer memecah kerjanya atau memindahkannya ke <code>worker_threads</code> .
Compatibility co-existence (desain bahasa)	Bersyarat	Semua script di satu halaman berbagi objek bawaan seperti <code>Array.prototype</code> yang bisa diubah siapa saja, jadi beberapa library hanya bisa hidup berdampingan kalau tidak saling mengubah objek bawaan. Pada SmooshGate (2018), <code>Array.prototype.flatten</code> di Firefox Nightly merusak situs yang memakai MooTools, sehingga TC39 mengganti namanya menjadi <code>flat</code> demi prinsip "don't break the Web".
Interaction Capability user error protection (desain bahasa)	Tidak	Desain JavaScript mengubah tipe data diam-diam alih-alih memunculkan error: <code>"5" + 9</code> menjadi <code>"59"</code> , dan <code>sort()</code> tanpa fungsi pembandingan mengurutkan angka sebagai string sehingga 100 ditaruh sebelum 85. Bahasanya tidak memberi tanda apa pun, jadi pencegahannya sepenuhnya bergantung pada disiplin programmer, misalnya selalu memakai <code>Number()</code> dan fungsi pembandingan.
Reliability faultlessness (desain bahasa)	Bersyarat	Karena tipenya dinamis, kesalahan seperti membaca properti dari <code>undefined</code> baru ketahuan saat baris itu dijalankan. Gao, Bird, dan Barr (ICSE 2017) menguji 400 bug yang sudah diperbaiki dari 398 proyek JavaScript dan menemukan Flow maupun TypeScript masing-masing bisa mendeteksi 15% di antaranya, padahal bug itu sudah lolos testing dan review.
Security integrity, resistance (desain bahasa)	Bersyarat	<code>Object.prototype</code> bisa diubah saat program berjalan, sehingga key dari input user bisa menyisipkan properti ke semua objek (prototype pollution). Contohnya CVE-2019-10744 (CVSS 9.1) pada fungsi <code>defaultsDeep</code> di <code>lodash</code> sebelum versi 4.17.12. Pertahanannya ada, misalnya <code>Object.create(null)</code> , <code>Map</code> , atau <code>Object.freeze(Object.prototype)</code> , tetapi harus dipasang sendiri oleh programmer.

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Maintainability modularity, reusability (ekosistem)	Bersyarat	npm membuat fungsi sekecil apa pun mudah dipakai ulang sebagai paket, tetapi kestabilan proyek jadi bergantung pada dependency milik orang lain. Pada Maret 2016 seorang developer menghapus 273 paketnya dari npm, termasuk left-pad yang ternyata ikut dipakai Babel secara tidak langsung, sehingga npm mencatat ratusan kegagalan per menit selama sekitar 2,5 jam.
Flexibility adaptability, installability (implementasi)	Bersyarat	Kode JavaScript bisa jalan di browser, di server lewat Node.js, dan di desktop lewat Electron, tetapi kode yang memakai API khusus satu lingkungan tidak otomatis bisa dipindah. Visual Studio Code versi desktop harus diinstal, sedangkan versinya di vscode.dev bisa langsung dibuka di browser sejak Oktober 2021, tetapi tanpa terminal dan debugger.
Safety fail safe, safe integration (desain bahasa)	Bersyarat	JavaScript mengatur memori lewat garbage collector yang waktunya tidak bisa dikendalikan dari dalam bahasa, padahal aturan Power of Ten NASA JPL melarang alokasi memori dinamis setelah inisialisasi. Karena itu keselamatan harus dijamin arsitektur: di SpaceX Crew Dragon (2020), JavaScript hanya dipakai untuk tampilan layar sentuh, sedangkan software penerbangan dan tombol fisik untuk aksi kritis berada di luar layar.

Kode 2.7: contoh untuk baris Functional Suitability pada JavaScript

```
const id = 10765432100123456789;
console.log(id.toString());           // "10765432100123458000", angkanya berubah
console.log(10765432100123456789n); // 10765432100123456789n, BigInt tetap utuh
```

Kode 2.8: contoh untuk baris Interaction Capability pada JavaScript

```
console.log("5" + 9);                  // "59", hasilnya string
console.log([90, 100, 85].sort());    // [ 100, 85, 90 ]
console.log([90, 100, 85].sort((a, b) => a - b)); // [ 85, 90, 100 ]
```

Rangkuman JavaScript: JavaScript paling cocok untuk produk yang memprioritaskan kemudahan diakses dari banyak perangkat, seperti aplikasi web yang langsung jalan di browser, tetapi tidak untuk lapisan kritis pada sistem yang menyangkut keselamatan.

2.5 SQL

SQL ada di level very high-level/4GL (slide 6) dan paradigma logic/declarative bersama Prolog (slide 9): programmer cukup menyebut data apa yang diinginkan, DBMS yang menentukan caranya. Karena dirancang khusus untuk mengelola data, SQL sekaligus termasuk domain-specific language, dan ekstensi prosedural seperti PL/pgSQL, PL/SQL, dan Transact-SQL membuatnya multi-paradigm secara terbatas (slide 12).

Tabel 2.5: Penilaian SQL terhadap ISO/IEC 25010:2023

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Functional Suitability functional completeness, functional appropriateness (desain bahasa)	Ya	Di domain data, semua tugas pokok sudah jadi perintah bawaan: membaca (<code>SELECT</code>), mengubah (<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>), membuat skema (<code>CREATE</code>), dan mengatur hak akses (<code>GRANT</code>). Rekursi pun tersedia sejak SQL:1999, jadi seluruh rantai prasyarat sebuah mata kuliah bisa ditelusuri dengan satu query <code>WITH RECURSIVE</code> tanpa loop di bahasa lain.
Performance Efficiency time behaviour, resource utilization (implementasi)	Bersyarat	Cara eksekusi query dipilih query optimizer, dan hasilnya bergantung pada indeks yang dibuat programmer. Pada skrip di bawah (dicoba di sqlite3 3.51.0), mencari satu NIM di antara 1.000.000 baris lewat <code>SCAN</code> (membaca seluruh tabel) makan sekitar 22 ms, sedangkan setelah <code>CREATE INDEX</code> berubah jadi <code>SEARCH</code> di bawah 1 ms, tetapi file database membengkak dari 29,2 MB ke 42,3 MB.
Compatibility interoperability (implementasi)	Bersyarat	Walau ada standar ISO/IEC 9075, dialek tiap vendor bisa mengubah arti query tanpa error: <code>SELECT 'A' 'B'</code> ; menghasilkan AB di PostgreSQL dan SQLite, tetapi 0 di MySQL 8.4, karena tanpa mode <code>PIPES_AS_CONCAT</code> operator <code> </code> di MySQL berarti OR logika.
Interaction Capability learnability, user error protection (desain bahasa)	Bersyarat	SQL dirancang agar analis bisnis yang bukan programmer bisa membaca query seperti kalimat bahasa Inggris, tetapi menurut The Register (2024) SQL akhirnya terutama dipakai programmer, administrator database, dan data scientist. Perlindungan dari kesalahan juga lemah: <code>DELETE FROM mahasiswa;</code> tanpa <code>WHERE</code> tetap sah dan menghapus semua baris, sedangkan penolakan lewat <code>sql_safe_updates</code> di MySQL bawaannya mati.
Reliability faultlessness, recoverability (desain bahasa)	Bersyarat	Bahasa SQL hanya menyediakan <code>COMMIT</code> dan <code>ROLLBACK</code> , sedangkan jaminan pulih setelah crash ada di mesin DBMS: InnoDB punya crash recovery, tetapi pada tabel MyISAM <code>ROLLBACK</code> hanya memunculkan peringatan dan perubahannya tetap tersimpan. Faultlessness juga rawan karena <code>NULL</code> memakai logika tiga nilai, jadi <code>NOT IN</code> pada kode di bawah diam-diam mengembalikan 0 baris, sampai wiki PostgreSQL menyarankan tidak memakainya.
Security confidentiality, resistance (desain bahasa)	Bersyarat	<code>GRANT</code> dan <code>REVOKE</code> menjaga kerahasiaan sampai per kolom di PostgreSQL, tetapi ketahanan terhadap serangan bergantung programmer karena parameterized query tidak diwajibkan, seperti pada kode di bawah. SQL injection di MOVEit Transfer (CVE-2023-34362, CVSS 9,8) dieksploitasi geng ransomware CL0P sejak 27 Mei 2023 untuk mencuri data dari database-nya.
Maintainability modularity, testability (ekosistem)	Bersyarat	CTE (subquery bernama di klausa <code>WITH</code>) membantu modularity karena query rumit bisa dipecah jadi bagian kecil. Begitu query tersebar di banyak file, urutan build dan tes otomatis diurus alat luar seperti dbt, lewat rujukan <code>{{ ref('model_a') }}</code> dan perintah <code>dbt test</code> untuk aturan <code>unique</code> atau <code>not_null</code> .
Flexibility adaptability, installability (implementasi)	Bersyarat	SQLite tidak butuh server maupun konfigurasi, file database-nya bisa disalin antarsistem 32/64-bit maupun big/little-endian, dan sudah tertanam di setiap Android, iPhone, Mac, serta Windows 10/11. Sebaliknya, PostgreSQL dan MySQL harus dipasang dan diinisialisasi dulu, dan query untuk satu dialek belum tentu jalan sama di dialek lain, seperti <code>'A' 'B'</code> di baris Compatibility.

Karakteristik (sub) dan lapisan	Vonis	Justifikasi dan kasus
Safety operational constraint, fail safe (implementasi)	Bersyarat	Batas aman bisa ditulis di skema, misalnya <code>CHECK (saldo >= 0)</code> , sehingga di SQLite <code>INSERT saldo -500</code> ditolak dan tabel tetap utuh (fail safe). Tapi penegakannya bergantung implementasi: MySQL sebelum 8.0.16 (2019) membaca <code>CHECK</code> lalu mengabaikannya, dan SQLite tetap menerima pelanggaran <code>FOREIGN KEY</code> kalau <code>PRAGMA foreign_keys = ON</code> tidak dijalankan di tiap koneksi.

Kode 2.9: contoh untuk baris Performance Efficiency pada SQL

```
-- sqlite3 uji.db < skrip.sql
CREATE TABLE mahasiswa(nim INTEGER, nama TEXT);
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i+1 FROM n WHERE i < 1000000)
INSERT INTO mahasiswa SELECT 241511000 + i, 'Mahasiswa ' || i FROM n;
.eqp on
.timer on
SELECT * FROM mahasiswa WHERE nim = 241911500; -- SCAN mahasiswa
SELECT page_count * page_size FROM pragma_page_count, pragma_page_size; -- 29212672
byte
CREATE INDEX idx_nim ON mahasiswa(nim);
SELECT * FROM mahasiswa WHERE nim = 241911500; -- SEARCH mahasiswa USING INDEX idx_
nim (nim=?)
SELECT page_count * page_size FROM pragma_page_count, pragma_page_size; -- 42287104
byte
```

Kode 2.10: contoh untuk baris Reliability pada SQL

```
SELECT 3 NOT IN (1, 2, NULL); -- NULL, bukan TRUE
SELECT nama FROM mhs WHERE nim NOT IN (SELECT nim FROM nilai); -- citra, yang belum
dinilai
INSERT INTO nilai VALUES (NULL, 70);
-- query NOT IN yang sama sekarang 0 baris, tanpa error
```

Kode 2.11: contoh untuk baris Security pada SQL

```
nama = "' OR '1'='1"
cur.execute("SELECT * FROM users WHERE nama = '" + nama + "'") # semua baris bocor
cur.execute("SELECT * FROM users WHERE nama = ?", (nama,)) # 0 baris, aman
```

Rangkuman SQL: SQL paling cocok untuk produk yang memprioritaskan penyimpanan dan pengambilan data terstruktur, dari sistem akademik kampus sampai aplikasi mobile yang datanya disimpan di perangkat, asalkan timnya disiplin memakai parameterized query dan constraint.

BAB III KESIMPULAN

3.1 Matriks Vonis

Tabel 3.1: Ringkasan vonis kelima bahasa

Karakteristik	C	Java	Python	JavaScript	SQL
Functional Suitability	Bersyarat	Bersyarat	Bersyarat	Bersyarat	Ya
Performance Efficiency	Ya	Bersyarat	Bersyarat	Bersyarat	Bersyarat
Compatibility	Ya	Bersyarat	Bersyarat	Bersyarat	Bersyarat
Interaction Capability	Tidak	Bersyarat	Ya	Tidak	Bersyarat
Reliability	Bersyarat	Bersyarat	Bersyarat	Bersyarat	Bersyarat
Security	Tidak	Bersyarat	Bersyarat	Bersyarat	Bersyarat
Maintainability	Bersyarat	Ya	Bersyarat	Bersyarat	Bersyarat
Flexibility	Bersyarat	Ya	Bersyarat	Bersyarat	Bersyarat
Safety	Bersyarat	Tidak	Bersyarat	Bersyarat	Bersyarat
Jumlah Ya / Bersyarat / Tidak	2 / 5 / 2	2 / 6 / 1	1 / 8 / 0	0 / 8 / 1	1 / 8 / 0

3.2 Jawaban atas Pertanyaan Tugas

Jadi, apakah kelima bahasa memenuhi kualitas pada Gambar 1? Tidak ada satu pun yang memenuhi kesembilan karakteristik tanpa syarat. Dari 45 penilaian, hanya 6 yang Ya, 35 Bersyarat, dan 4 Tidak. Vonis Ya muncul ketika desain atau implementasi resmi memang memaksakan sesuatu, seperti tipe statis Java yang menahan perubahan berbahaya saat kompilasi. Vonis Tidak muncul ketika desain bahasa justru meloloskan kesalahan, seperti C yang tidak mengecek batas buffer atau JavaScript yang mengubah tipe data diam-diam.

Banyaknya vonis Bersyarat bukan tanda penilaian yang ragu-ragu. Syaratnya hampir selalu jatuh di luar desain bahasa: indeks di SQL, venv di Python, library JSON di Java, atau pemeriksa tipe seperti TypeScript di JavaScript. Ini sejalan dengan subbab 1.3. ISO/IEC 25010 menilai produk, sedangkan bahasa hanya menentukan seberapa mudah produk yang berkualitas bisa dibuat.

3.3 Pelajaran untuk Tugas Akhir DSL

Pola ini penting untuk tugas akhir mata kuliah, yaitu membuat domain specific language. SQL adalah contoh DSL paling nyata di tugas ini, dan satu-satunya vonis Ya untuk Functional Suitability justru jatuh ke bahasa dengan domain paling sempit, karena generality sengaja dikorbankan demi fungsi yang lengkap di domain data. Kelemahannya juga jadi pelajaran. SQL injection dan `DELETE` tanpa `WHERE` terjadi karena bahasanya membiarkan perintah berbahaya tetap sah, jadi DSL yang dirancang nanti sebaiknya mencegah kesalahan seperti itu sejak tata bahasanya, bukan menyerahkannya ke disiplin pengguna.

DAFTAR PUSTAKA

Umum

- [1] ISO/IEC 25010:2023. Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE), Product quality model. ISO, Jenewa.
- [2] ISO/IEC 9126-1:2001. Software engineering: Product quality, Part 1: Quality model. ISO, Jenewa.
- [3] R. W. Sebesta. Concepts of Programming Languages, edisi 12. Pearson, 2019.
- [4] Materi kuliah Programming Language Pragmatics, Suplemen 1 PLP 2026, slide 6, 9, 12, dan 23 sampai 27. Politeknik Negeri Bandung.

C

- [5] D. M. Ritchie. The Development of the C Language.
<https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.html>
- [6] K. Thompson. UNIX Implementation. Bell System Technical Journal 57(6), 1978.
<https://archive.org/details/bstj57-6-1931>
- [7] R. Pereira dkk. Energy Efficiency across Programming Languages. SLE 2017.
<https://greenlab.di.uminho.pt/wp-content/uploads/2017/10/sleFinal.pdf>
- [8] SQLite. Why Is SQLite Coded In C. <https://www.sqlite.org/whyc.html>
- [9] LWN.net. An attempt to backdoor the kernel (2003). <https://lwn.net/Articles/57135/>
- [10] RISKS Digest 9.69, laporan gangguan jaringan AT&T 15 Januari 1990. <https://catless.ncl.ac.uk/risks/9/69>
- [11] NVD. CVE-2014-0160 (Heartbleed). <https://nvd.nist.gov/vuln/detail/CVE-2014-0160>
- [12] NSA. Software Memory Safety (2022), ringkasan WA Cyber Security Unit.
<https://soc.cyber.wa.gov.au/advisories/20221124001-NSA-Software-Memory-Safety/>
- [13] GNU CPP Manual. Operator Precedence Problems.
<https://gcc.gnu.org/onlinedocs/cpp/Operator-Precedence-Problems.html>
- [14] Microsoft Learn. Data Type Ranges. <https://learn.microsoft.com/en-us/cpp/cpp/data-type-ranges>
- [15] G. J. Holzmann. The Power of 10: Rules for Developing Safety-Critical Code. IEEE Computer, 2006.
<https://spinroot.com/gerard/pdf/P10.pdf>

Java

- [16] OpenJDK. JDK-5045582, overflow pada binarySearch. <https://bugs.openjdk.org/browse/JDK-5045582>
- [17] J. Bloch. Nearly All Binary Searches and Mergesorts are Broken (2006).
<https://research.google/blog/extra-extra-read-all-about-it-nearly-all-binary-searches-and-mergesorts-are-broken/>
- [18] G. Morling. 1BRC Results Are In (2024). <https://www.morling.dev/blog/1brc-results-are-in/>
- [19] OpenJDK. JEP 321: HTTP Client. <https://openjdk.org/jeps/321>
- [20] OpenJDK. JEP 540, API JSON (incubator). <https://openjdk.org/jeps/540>
- [21] OpenJDK. JEP 512: Compact Source Files and Instance Main Methods. <https://openjdk.org/jeps/512>
- [22] Meta Engineering. Retrofitting null-safety onto Java at Meta (2022).
<https://engineering.fb.com/2022/11/22/developer-tools/meta-java-nullsafe/>
- [23] Apache Logging Services. Log4j Security Vulnerabilities. <https://logging.apache.org/security.html>
- [24] Mojang. Important Message: Security Vulnerability in Java Edition (2021).
<https://www.minecraft.net/en-us/article/important-message--security-vulnerability-java-edition>
- [25] OpenJDK. JEP 395: Records. <https://openjdk.org/jeps/395>
- [26] Oracle. The Java Virtual Machine Specification SE 25, bagian 2.1.
<https://docs.oracle.com/javase/specs/jvms/se25/html/jvms-2.html>
- [27] JCP. JSR 302: Safety Critical Java Technology. <https://jcp.org/en/jsr/detail?id=302>

Python

- [28] Python Tutorial. Floating-Point Arithmetic: Issues and Limitations.
<https://docs.python.org/3/tutorial/float.html>
- [29] PEP 703: Making the Global Interpreter Lock Optional in CPython. <https://peps.python.org/pep-0703/>

- [30] PEP 668: Marking Python base environments as externally managed. <https://peps.python.org/pep-0668/>
- [31] What's New in Python 3.10, 3.12, dan 3.14, bagian pesan error. <https://docs.python.org/3/whatsnew/3.14.html>
- [32] Python Tutorial. Default Argument Values.
<https://docs.python.org/3/tutorial/controlflow.html#default-argument-values>
- [33] The Register. 15-year-old Python flaw found in over 350,000 projects (2022).
<https://www.theregister.com/security/2022/09/22/15-year-old-python-flaw-found-in-over-350000-projects/1189100>
- [34] PEP 706: Filter for tarfile.extractall. <https://peps.python.org/pep-0706/>
- [35] Dropbox. Our journey to type checking 4 million lines of Python (2019).
<https://dropbox.tech/application/our-journey-to-type-checking-4-million-lines-of-python>
- [36] PEP 597: Add optional EncodingWarning. <https://peps.python.org/pep-0597/>
- [37] PEP 686: Make UTF-8 mode default. <https://peps.python.org/pep-0686/>
- [38] Python Language Reference. The assert statement.
https://docs.python.org/3/reference/simple_stmts.html#the-assert-statement

JavaScript

- [39] X Developer Platform. X IDs. <https://docs.x.com/fundamentals/x-ids>
- [40] Node.js. Don't Block the Event Loop. <https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop>
- [41] Chrome for Developers. #SmoooshGate FAQ (2018). <https://developer.chrome.com/blog/smooshgate>
- [42] MDN. Array.prototype.sort().
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort
- [43] Z. Gao, C. Bird, E. T. Barr. To Type or Not to Type: Quantifying Detectable Bugs in JavaScript. ICSE 2017.
<https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/gao2017javascript.pdf>
- [44] NVD. CVE-2019-10744 (lodash). <https://nvd.nist.gov/vuln/detail/CVE-2019-10744>
- [45] npm Blog. kik, left-pad, and npm (2016). <https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm>
- [46] Visual Studio Code Blog. vscode.dev (2021). <https://code.visualstudio.com/blogs/2021/10/20/vscode-dev>
- [47] SpaceX Software Team. Reddit AMA r/spacex (Juni 2020).
https://www.reddit.com/r/spacex/comments/gxb7jl/we_are_the_spacex_software_team_ask_us_anything/ft68jpb/

SQL

- [48] PostgreSQL Documentation. WITH Queries (Common Table Expressions).
<https://www.postgresql.org/docs/current/queries-with.html>
- [49] SQLite. EXPLAIN QUERY PLAN. <https://www.sqlite.org/eqp.html>
- [50] MySQL 8.4 Reference Manual. Logical Operators. <https://dev.mysql.com/doc/refman/8.4/en/logical-operators.html>
- [51] The Register (31 Mei 2024), 50 tahun SQL bersama Donald Chamberlin.
https://www.theregister.com/2024/05/31/fifty_years_of_sql/
- [52] MySQL 8.4 Reference Manual. Server System Variables (sql_safe_updates).
<https://dev.mysql.com/doc/refman/8.4/en/server-system-variables.html>
- [53] MySQL 8.4 Reference Manual. START TRANSACTION, COMMIT, and ROLLBACK Statements.
<https://dev.mysql.com/doc/refman/8.4/en/commit.html>
- [54] PostgreSQL Wiki. Don't Do This. https://wiki.postgresql.org/wiki/Don%27t_Do_This
- [55] NVD. CVE-2023-34362 (MOVEit Transfer). <https://nvd.nist.gov/vuln/detail/CVE-2023-34362>
- [56] CISA. Advisory AA23-158A, CL0P dan MOVEit.
<https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-158a>
- [57] dbt Documentation. Data tests. <https://docs.getdbt.com/docs/build/data-tests>
- [58] SQLite. Most Widely Deployed SQL Database Engine. <https://www.sqlite.org/mostdeployed.html>
- [59] MySQL 8.0 Release Notes. Changes in MySQL 8.0.16.
<https://dev.mysql.com/doc/relnotes/mysql/8.0/en/news-8-0-16.html>
- [60] SQLite. Foreign Key Support. <https://www.sqlite.org/foreignkeys.html>